

ScholarVerse

An Autonomous Multi-Agent System for Scientific Research Analysis



Overview

ScholarVerse is a cutting-edge multi-agent system that transforms how researchers interact with scientific literature. By orchestrating specialized AI agents through Google's Agent Development Kit (ADK), ScholarVerse ingests scientific papers, constructs knowledge graphs, generates cross-paper insights, verifies claims, and presents findings through interactive 3D visualizations.

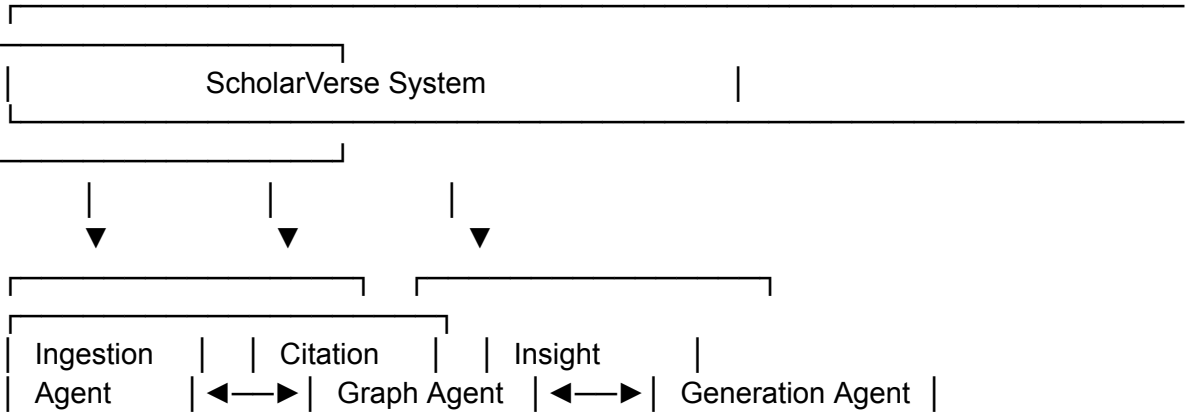
Key Features:

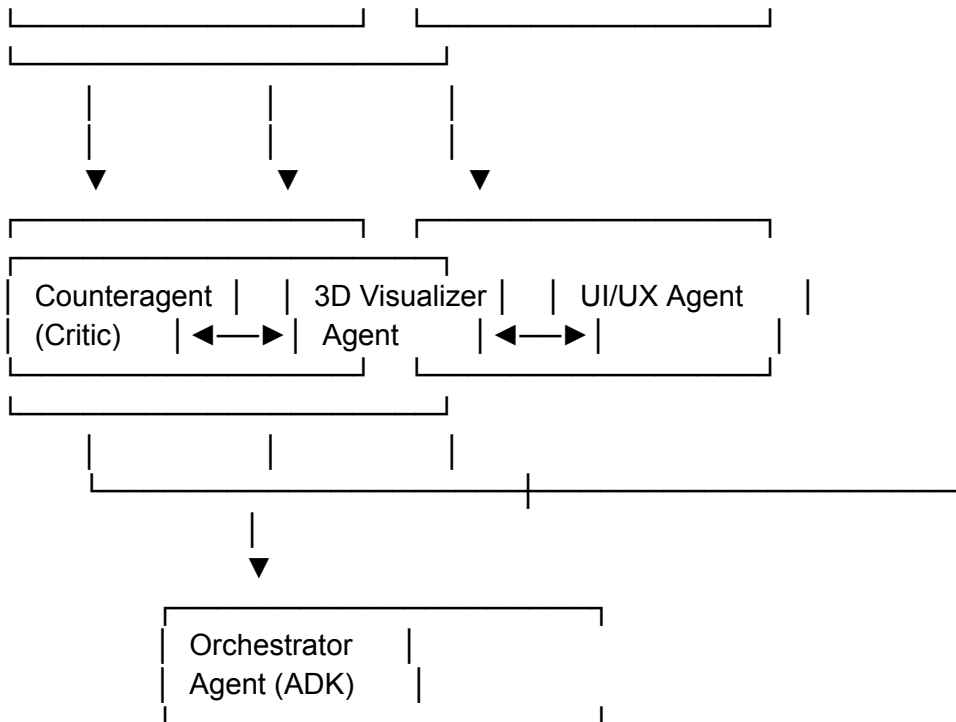
-  Automated PDF ingestion and structured content extraction
-  Dynamic citation and knowledge graph construction
-  Cross-paper trend analysis and insight generation
-  Autonomous claim verification and fact-checking
-  Interactive 3D visualization of research networks
-  Real-time dashboard for research exploration

System Architecture

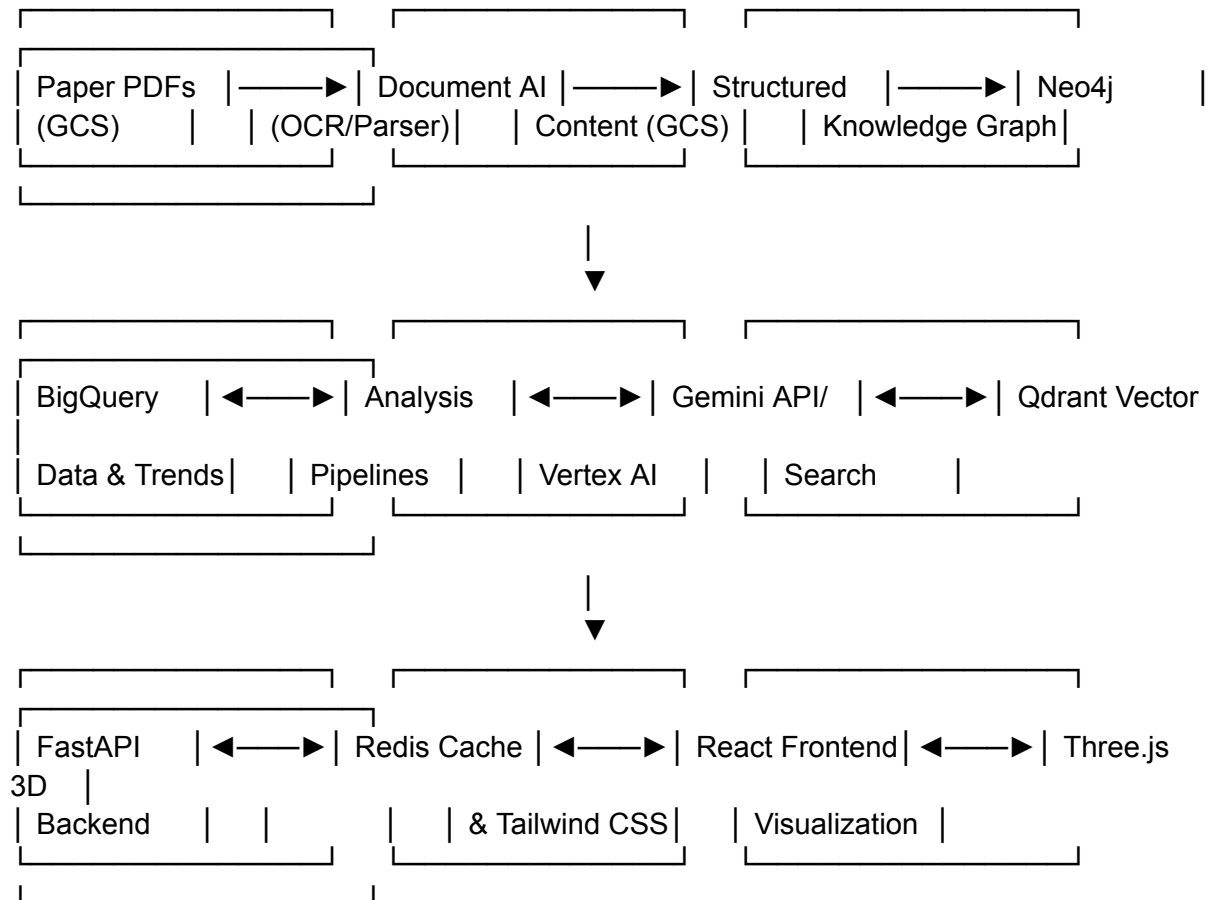
ScholarVerse employs a microservices architecture deployed on Google Cloud Platform, utilizing several specialized agents that communicate through the ADK protocol.

Agent Architecture Overview





Data Flow Architecture



Agent Roles and Capabilities

Ingestion Agent

- **Purpose:** Extract structured content from PDFs
- **Technologies:** Document AI, Vision OCR, SciSpacy, Florence/Vision models
- **Functions:**
 - Extract text, tables, and figure metadata
 - Parse document structure (titles, abstracts, sections)
 - Identify inline citations and references
 - Store extracted content for downstream processing

Citation Graph Agent

- **Purpose:** Build knowledge graph of publications and concepts
- **Technologies:** Neo4j, embeddings models, graph algorithms
- **Functions:**
 - Parse metadata (authors, titles, references)
 - Construct graph with papers, authors, institutions, and concepts
 - Compute graph metrics (centrality, communities)
 - Generate embeddings for semantic search

Insight Generation Agent

- **Purpose:** Generate cross-paper insights and trends
- **Technologies:** BigQuery, Gemini API/Vertex AI
- **Functions:**
 - Query BigQuery for aggregate trends
 - Analyze citation patterns and topic evolution
 - Identify emerging topics and benchmark trends
 - Synthesize findings into coherent insights

Counteragent (Critic)

- **Purpose:** Verify claims and reduce hallucinations
- **Technologies:** Vertex AI, BigQuery, external databases
- **Functions:**
 - Verify generated claims against source material
 - Challenge biased or unsupported statements
 - Provide alternative interpretations of data
 - Implement fallback mechanisms for uncertain results

3D Visualizer Agent

- **Purpose:** Create interactive visualizations of research networks
- **Technologies:** Three.js, WebGL, WebSockets
- **Functions:**

- Render 3D force-directed knowledge graphs
- Implement real-time updates via WebSockets
- Enable interactive exploration (rotation, zoom, filtering)
- Visualize temporal trends and citation patterns

UI/UX Agent

- **Purpose:** Build user interfaces and reports
- **Technologies:** React, Tailwind CSS, Chart.js
- **Functions:**
 - Generate interactive dashboards
 - Display insights, charts, and tables
 - Support filtering by domain
 - Export reports in various formats

Orchestrator Agent

- **Purpose:** Coordinate workflow between agents
- **Technologies:** ADK, Cloud Run, Kubernetes
- **Functions:**
 - Dispatch tasks to appropriate agents
 - Manage sequential and parallel workflows
 - Handle error cases and fallbacks
 - Monitor system performance

Project Structure

```

scholar-verse/
├── .github/           # GitHub workflows and CI/CD
├── architecture/     # Architecture diagrams and documentation
├── deployment/       # Deployment configuration
├── terraform/        # Infrastructure as code
├── docker/           # Docker configuration
├── kubernetes/       # Kubernetes manifests
├── frontend/         # UI components
│   ├── public/       # Static assets
│   └── src/          # React components
│       ├── components/ # Reusable UI components
│       ├── pages/     # Page definitions
│       ├── visualizations/ # Visualization components
│       │   ├── threed/ # Three.js visualization components
│       │   └── charts/ # Chart.js data visualizations
│       ├── hooks/     # Custom React hooks
│       ├── state/     # State management
│       └── utils/     # Utility functions
├── tests/            # Frontend tests
├── backend/          # Backend services
└── api/              # FastAPI application
  
```

```

├── routes/           # API endpoint definitions
├── models/           # Data models
├── services/         # Business logic
├── utils/            # Utility functions
├── tests/            # Backend tests
├── agents/           # Agent implementations
│   ├── common/       # Shared agent utilities
│   │   ├── messaging/ # Agent communication protocols
│   │   ├── models/    # Shared data models
│   │   └── utils/      # Common utilities
│   ├── orchestrator/ # Orchestrator agent
│   ├── ingestion/     # Document ingestion agent
│   │   ├── extractors/ # Content extraction modules
│   │   ├── processors/ # Document processing pipelines
│   │   └── models/     # Document models
│   ├── citation_graph/ # Citation graph agent
│   │   ├── graph_builder/ # Graph construction logic
│   │   ├── algorithms/    # Graph algorithms
│   │   └── query/         # Graph query utilities
│   ├── insight/        # Insight generation agent
│   │   ├── analyzers/    # Analysis modules
│   │   ├── queries/      # BigQuery templates
│   │   └── generators/   # Insight generation logic
│   ├── counter/        # Counteragent/critic
│   │   ├── verification/ # Claim verification logic
│   │   ├── fallbacks/    # Fallback mechanisms
│   │   └── models/       # Verification models
│   ├── visualizer/     # 3D visualizer agent
│   │   ├── renderers/    # Visualization renderers
│   │   ├── layouts/      # Graph layout algorithms
│   │   └── streaming/    # WebSocket streaming logic
│   └── ui/             # UI/UX agent
│       ├── generators/   # UI generation logic
│       └── templates/    # UI templates
├── data/             # Data processing utilities
│   ├── schemas/         # Data schemas
│   ├── pipelines/       # Data processing pipelines
│   ├── connectors/      # Database connectors
│   └── embeddings/      # Embedding models
├── scripts/          # Utility scripts
├── docs/             # Documentation
├── tests/            # Integration tests
├── README.md         # This file
├── requirements.txt   # Python dependencies
├── package.json       # Node.js dependencies
└── docker-compose.yml # Local development setup

```

Implementation Phases

Phase 1: Infrastructure Setup and Core Components (Weeks 1-2)

Week 1: Initial Setup and Infrastructure

- Set up Git repository and project structure
- Configure development environment and tools
- Deploy basic cloud infrastructure via Terraform
- Implement core data models and schemas
- Set up CI/CD pipelines

Week 2: Data Pipeline and Storage

- Implement document storage in Google Cloud Storage
- Set up Document AI integration for OCR and parsing
- Configure Neo4j graph database
- Deploy Qdrant for vector search
- Implement basic data connectors

Phase 2: Agent Development (Weeks 3-6)

Week 3: Ingestion Agent

- Implement Document AI integration
- Build PDF text and structure extraction
- Develop figure and table detection
- Create metadata extraction pipeline
- Set up content storage and indexing

Week 4: Citation Graph Agent

- Implement citation parsing and normalization
- Build knowledge graph construction logic
- Develop entity resolution for papers and authors
- Set up graph algorithms for metrics computation
- Create embedding generation pipeline

Week 5: Insight and Counter Agents

- Implement BigQuery integration for trend analysis
- Set up Gemini API/Vertex AI for insight generation
- Build claim verification mechanisms
- Develop fallback strategies
- Create insight aggregation logic

Week 6: Visualization and UI Agents

- Implement Three.js visualization components

- Build WebSocket streaming for real-time updates
- Develop React dashboard components
- Create report generation functionality
- Implement filtering and search capabilities

Phase 3: Integration and Orchestration (Weeks 7-8)

Week 7: Agent Orchestration

- Implement ADK integration
- Build orchestrator agent
- Set up agent communication protocols
- Develop workflow patterns (sequential, parallel, loop)
- Implement error handling and recovery

Week 8: System Integration

- Integrate all agent components
- Implement end-to-end workflows
- Set up monitoring and logging
- Optimize performance
- Document APIs and interfaces

Phase 4: Testing, Optimization, and Deployment (Weeks 9-10)

Week 9: Testing and Optimization

- Perform unit and integration testing
- Conduct load and stress testing
- Optimize performance bottlenecks
- Implement caching strategies
- Refine user experience

Week 10: Deployment and Documentation

- Deploy to production environment
- Create comprehensive documentation
- Prepare demo materials
- Develop user guides
- Finalize project presentation



Technical Implementation Details

Cloud Infrastructure

Google Cloud Resources

- **Google Cloud Storage**
 - `scholarverse-pdfs`: Raw PDF storage
 - `scholarverse-extracted`: Structured content
 - `scholarverse-artifacts`: Generated insights and reports
- **BigQuery Datasets**
 - `scholarverse-metadata`: Paper metadata and citations
 - `scholarverse-content`: Extracted content and insights
 - `scholarverse-analytics`: Usage and performance metrics
- **Compute Resources**
 - Agent containers deployed on Cloud Run or GKE
 - FastAPI backend on Cloud Run
 - Frontend hosted on Firebase Hosting
- **AI/ML Resources**
 - Document AI processors for OCR and parsing
 - Vertex AI endpoints for Gemini API access
 - Custom models deployed on Vertex AI (optional)

Networking

- Cloud VPC with private networking for internal services
- Cloud NAT for outbound connectivity
- Cloud Load Balancing for frontend and API endpoints
- Private Service Connect for database access

Data Models

Paper Document

```
{
  "id": "paper_123",
  "title": "Multi-Agent Systems for Scientific Literature Analysis",
  "abstract": "This paper presents a novel approach...",
  "authors": [
    {
      "name": "Jane Smith",
      "affiliation": "University of AI"
    }
  ],
  "publication_date": "2023-05-15",
  "venue": "International Conference on AI",
  "doi": "10.1234/5678",
  "sections": [
    {
      "title": "Introduction",

```



```

    "content": "In recent years, the volume of scientific literature..."
  }
],
"figures": [
  {
    "id": "fig1",
    "caption": "Architecture of proposed system",
    "image_url": "gs://scholarverse-extracted/paper_123/figures/fig1.png"
  }
],
"tables": [],
"references": [
  {
    "id": "ref1",
    "text": "Smith, J. (2022). Knowledge Graphs for Scientific Literature."
  }
],
"citations": [
  {
    "reference_id": "ref1",
    "context": "Knowledge graphs have been shown to improve literature analysis [1]."
  }
]
}

```

Knowledge Graph Schema

// Neo4j Schema

// Nodes

```

CREATE (p:Paper {
  id: String,
  title: String,
  abstract: String,
  publication_date: Date,
  doi: String,
  embedding: List<Float>
})

```

```

CREATE (a:Author {
  id: String,
  name: String,
  affiliation: String
})

```

```

CREATE (i:Institution {
  id: String,
  name: String,

```

```
location: String
})
```

```
CREATE (c:Concept {
  id: String,
  name: String,
  description: String,
  embedding: List<Float>
})
```

```
// Relationships
```

```
CREATE (a:Author)-[:AUTHORED]->(p:Paper)
CREATE (a:Author)-[:AFFILIATED_WITH]->(i:Institution)
CREATE (p1:Paper)-[:CITES {context: String}]->(p2:Paper)
CREATE (p:Paper)-[:MENTIONS {relevance: Float}]->(c:Concept)
CREATE (c1:Concept)-[:RELATED_TO {strength: Float}]->(c2:Concept)
```

Agent Implementation Details

Ingestion Agent

The Ingestion Agent uses a pipeline architecture to process PDFs:

1. Document Preprocessing

- File validation and virus scanning
- PDF metadata extraction
- Page segmentation

2. Text Extraction

- OCR via Document AI
- Layout analysis
- Text block identification

3. Structure Analysis

- Section identification
- Header/footer detection
- Reference list extraction

4. Entity Extraction

- Scientific entity recognition (using SciSpacy)
- Citation marker identification
- Figure and table detection

Sample Code: Document AI Integration

```
from google.cloud import documentai_v1 as documentai
```

```

def process_document(project_id, location, processor_id, file_path, mime_type):
    client = documentai.DocumentProcessorServiceClient()
    name = f"projects/{project_id}/locations/{location}/processors/{processor_id}"

    with open(file_path, "rb") as f:
        document_content = f.read()

    document = {"content": document_content, "mime_type": mime_type}
    request = {"name": name, "raw_document": document}

    result = client.process_document(request=request)
    document = result.document

    return document

```

Citation Graph Agent

The Citation Graph Agent constructs a knowledge graph using the following steps:

1. Reference Parsing

- Extract structured data from reference lists
- Normalize citation formats
- Match references to known papers

2. Entity Resolution

- Resolve author name variations
- Match institutions and affiliations
- Identify duplicate papers

3. Graph Construction

- Create nodes for papers, authors, institutions
- Establish citation relationships
- Link papers to concepts

4. Graph Analysis

- Compute centrality measures
- Identify communities
- Calculate similarity metrics

Sample Code: Neo4j Graph Construction

```

from neo4j import GraphDatabase

```

```

class CitationGraphBuilder:
    def __init__(self, uri, user, password):

```

```

self.driver = GraphDatabase.driver(uri, auth=(user, password))

def add_paper(self, paper_data):
    with self.driver.session() as session:
        result = session.write_transaction(self._create_paper_node, paper_data)
        return result

def add_citation(self, source_id, target_id, context):
    with self.driver.session() as session:
        result = session.write_transaction(
            self._create_citation_relationship, source_id, target_id, context)
        return result

@staticmethod
def _create_paper_node(tx, paper_data):
    query = """
    CREATE (p:Paper {
        id: $id,
        title: $title,
        abstract: $abstract,
        publication_date: date($pub_date)
    })
    RETURN p.id AS paper_id
    """
    result = tx.run(query,
        id=paper_data["id"],
        title=paper_data["title"],
        abstract=paper_data.get("abstract", ""),
        pub_date=paper_data.get("publication_date"))
    return result.single()["paper_id"]

@staticmethod
def _create_citation_relationship(tx, source_id, target_id, context):
    query = """
    MATCH (source:Paper {id: $source_id})
    MATCH (target:Paper {id: $target_id})
    CREATE (source)-[r:CITES {context: $context}]->(target)
    RETURN type(r) AS relationship_type
    """
    result = tx.run(query, source_id=source_id, target_id=target_id, context=context)
    return result.single()["relationship_type"]

```

Insight Generation Agent

The Insight Generation Agent leverages BigQuery and Gemini API to generate insights:

1. Query Generation

- Formulate BigQuery SQL for trend analysis
- Create parameterized queries based on user context
- Design efficient query patterns

2. Data Aggregation

- Execute queries against BigQuery datasets
- Aggregate results across multiple papers
- Compute statistical measures

3. Insight Synthesis

- Generate insights using Gemini API
- Contextualize findings with citations
- Identify patterns and trends

4. Output Formatting

- Structure insights for presentation
- Generate visualizations
- Prepare report content

Sample Code: BigQuery Integration

```
from google.cloud import bigquery
```

```
def analyze_citation_trends(dataset_id, years=5):  
    client = bigquery.Client()
```

```
    query = f"""  
    SELECT  
        EXTRACT(YEAR FROM p.publication_date) AS year,  
        COUNT(c.citing_paper_id) AS citation_count,  
        AVG(p.impact_score) AS avg_impact  
    FROM  
        `{dataset_id}.papers` p  
    JOIN  
        `{dataset_id}.citations` c  
    ON  
        p.paper_id = c.cited_paper_id  
    WHERE  
        EXTRACT(YEAR FROM p.publication_date) >= EXTRACT(YEAR FROM  
CURRENT_DATE()) - {years}  
    GROUP BY  
        year  
    ORDER BY  
        year ASC  
    """
```

```
query_job = client.query(query)
results = query_job.result()

return [dict(row) for row in results]
```

Sample Code: Gemini API Integration

```
from vertexai.generative_models import GenerativeModel, Part

def generate_insights(trend_data, context_papers):
    model = GenerativeModel("gemini-pro")

    context = f"""
    Based on the following citation trend data:
    {trend_data}

    And considering these key papers:
    {context_papers}

    Generate insights about research trends, identifying emerging topics,
    methodological shifts, and potential future directions.
    """

    response = model.generate_content(
        context,
        generation_config={
            "max_output_tokens": 1024,
            "temperature": 0.2,
            "top_p": 0.8,
            "top_k": 40
        }
    )

    return response.text
```

3D Visualizer Agent

The 3D Visualizer Agent creates interactive visualizations using Three.js:

1. Layout Generation

- Compute force-directed layout for graph
- Apply clustering algorithms
- Generate coordinates for nodes

2. Rendering

- Create Three.js scene
- Apply materials and textures
- Implement lighting and effects

3. Interaction

- Enable camera controls
- Implement picking and selection
- Support filtering and highlighting

4. Streaming

- Set up WebSocket connections
- Stream updates for real-time visualization
- Optimize data transfer

Sample Code: Three.js Visualization

```
import * as THREE from 'three';
import { ForceGraph3D } from '3d-force-graph';

class ResearchGraphVisualizer {
  constructor(containerId) {
    this.container = document.getElementById(containerId);
    this.graph = ForceGraph3D()(this.container)
      .nodeColor(node => this.getNodeColor(node))
      .nodeLabel(node => `${node.title}\nAuthors: ${node.authors}\nCitations:
${node.citations}`)
      .linkDirectionalParticles(3)
      .linkDirectionalParticleSpeed(0.005)
      .linkDirectionalParticleWidth(2)
      .onNodeClick(this.handleNodeClick.bind(this));

    this.initWebSocket();
  }

  getNodeColor(node) {
    const topicColors = {
      'AI': '#ff4500',
      'Healthcare': '#4287f5',
      'Legal': '#42f590',
      'Education': '#f5d142'
    };
    return topicColors[node.primaryTopic] || '#cccccc';
  }

  handleNodeClick(node) {
    // Center and highlight the node
    this.graph.centerAt(node.x, node.y, node.z, 1000);
    this.graph.cameraPosition(
```

```

    { x: node.x + 100, y: node.y, z: node.z + 100 },
    node,
    3000
  );

  // Emit selection event
  this.onNodeSelected(node);
}

updateData(graphData) {
  this.graph.graphData(graphData);
}

initWebSocket() {
  this.socket = new WebSocket('wss://api.scholarverse.example/graph-updates');
  this.socket.onmessage = (event) => {
    const update = JSON.parse(event.data);
    if (update.type === 'node_added') {
      this.addNode(update.data);
    } else if (update.type === 'link_added') {
      this.addLink(update.data);
    }
  };
}

addNode(node) {
  const { nodes, links } = this.graph.graphData();
  this.graph.graphData({
    nodes: [...nodes, node],
    links
  });
}

addLink(link) {
  const { nodes, links } = this.graph.graphData();
  this.graph.graphData({
    nodes,
    links: [...links, link]
  });
}

onNodeSelected(node) {
  // To be implemented by consumer
  console.log('Node selected:', node);
}
}

```


ADK Integration

The Orchestrator Agent uses Google's Agent Development Kit to coordinate all other agents:

1. Task Definition

- Define agent tasks and capabilities
- Set up agent communication patterns
- Configure agent tools and memory

2. Workflow Orchestration

- Implement sequential processing pipelines
- Set up parallel task execution
- Create iterative refinement loops

3. Error Handling

- Implement fallback strategies
- Set up retry mechanisms
- Configure graceful degradation

4. Monitoring

- Track agent performance
- Log agent activities
- Monitor resource utilization

Sample Code: ADK Orchestrator

```
from google.cloud.adk import (
    Agent, Configuration, EndpointHandler,
    SequentialAgent, LoopAgent, GeneratorCriticAgent
)

def create_orchestrator_agent():
    # Define agent capabilities
    ingestion_agent = Agent(
        name="ingestion_agent",
        endpoint="https://ingestion-agent.scholarverse.example/process",
        capabilities=["process_pdf", "extract_content"]
    )

    citation_graph_agent = Agent(
        name="citation_graph_agent",
        endpoint="https://citation-graph.scholarverse.example/graph",
        capabilities=["build_graph", "analyze_citations"]
    )

    insight_agent = Agent(
        name="insight_agent",
```

```

        endpoint="https://insight-agent.scholarverse.example/analyze",
        capabilities=["generate_insights", "analyze_trends"]
    )

    counter_agent = Agent(
        name="counter_agent",
        endpoint="https://counter-agent.scholarverse.example/verify",
        capabilities=["verify_claims", "provide_alternatives"]
    )

    # Create sequential pipeline for document processing
    document_pipeline = SequentialAgent(
        name="document_pipeline",
        agents=[ingestion_agent, citation_graph_agent]
    )

    # Create generator-critic pattern for insights
    insight_pipeline = GeneratorCriticAgent(
        name="insight_pipeline",
        generator=insight_agent,
        critic=counter_agent
    )

    # Create loop for iterative refinement
    refinement_loop = LoopAgent(
        name="refinement_loop",
        agent=insight_pipeline,
        condition="confidence_score >= 0.8 OR iterations >= 3"
    )

    # Create the orchestrator configuration
    config = Configuration(
        name="scholarverse_orchestrator",
        description="Orchestrates the ScholarVerse multi-agent system",
        agents=[
            document_pipeline,
            refinement_loop
        ]
    )

    return config

```

API Documentation

FastAPI Backend Endpoints

Document Management

- `POST /api/documents`
 - Upload new documents to the system
 - Request: `multipart/form-data` with PDF files
 - Response: List of document IDs and status
- `GET /api/documents/{document_id}`
 - Retrieve document metadata and status
 - Response: Document metadata
- `GET /api/documents/{document_id}/content`
 - Retrieve structured content of document
 - Response: Document content

Graph API

- `GET /api/graph`
 - Retrieve the entire knowledge graph
 - Query params: `limit`, `filter_topic`
 - Response: Graph data (nodes and edges)
- `GET /api/graph/nodes/{node_id}`
 - Retrieve specific node data
 - Response: Node details
- `GET /api/graph/search`
 - Search for nodes in the graph
 - Query params: `q`, `type`, `limit`
 - Response: Matching nodes

Insights API

- `GET /api/insights`
 - Retrieve generated insights
 - Query params: `topic`, `limit`, `start_date`, `end_date`
 - Response: List of insights
- `POST /api/insights/generate`
 - Generate new insights from specific documents
 - Request: List of document IDs
 - Response: Generated insights

WebSocket Endpoints

- `ws://api.scholarverse.example/graph-updates`
 - Stream of real-time graph updates
 - Messages: Node additions, edge creations, attribute updates
- `ws://api.scholarverse.example/insights-stream`
 - Stream of newly generated insights
 - Messages: New insights as they are created

Security Considerations

Authentication and Authorization

- Google Cloud IAM for service-to-service authentication
- Firebase Authentication for user authentication
- Role-based access control for different user types

Data Protection

- Encryption at rest for all storage
- TLS encryption for data in transit
- VPC Service Controls for network isolation

Compliance

- GDPR considerations for user data
- Academic data use policies
- Citation and attribution requirements

Deployment Instructions

Local Development Setup

Clone the repository

```
git clone https://github.com/yourusername/scholar-verse.git
cd scholar-verse
```

1.

Set up environment variables

```
cp .env.example .env
# Edit .env with your configuration
```

2.

Start local development environment with Docker Compose

```
docker-compose up -d
```

3.

4. Access the application

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:8000`
- Neo4j Browser: `http://localhost:7474`

Google Cloud Deployment

Configure Google Cloud project

```
gcloud config set project your-project-id
```

1.

Deploy infrastructure with Terraform

```
cd deployment/terraform
terraform init
terraform apply
```

2.

Deploy agents to Cloud Run

```
cd deployment/docker
./deploy_agents.sh
```

3.

Deploy frontend to Firebase Hosting

```
cd frontend
npm run build
firebase deploy
```

4.

Testing

Unit Testing

- Run agent unit tests: `pytest agents/*/tests/`

- Run frontend unit tests: `cd frontend && npm test`

Integration Testing

- Run integration tests: `pytest tests/integration/`

Performance Testing

- Load test API endpoints: `locust -f tests/performance/locustfile.py`
- Benchmark graph database performance: `python scripts/benchmark_neo4j.py`
- Profile agent performance: `python scripts/profile_agents.py`

Implementation Checklists

Infrastructure Setup Checklist

- ☐ **Google Cloud Project**
 - ☐ Create project and enable billing
 - ☐ Enable required APIs:
 - ☐ Document AI API
 - ☐ BigQuery API
 - ☐ Cloud Storage API
 - ☐ Vertex AI API
 - ☐ Kubernetes Engine API (if using GKE)
 - ☐ Cloud Run API
 - ☐ Set up IAM roles and service accounts
- ☐ **Data Storage**
 - ☐ Create Cloud Storage buckets
 - ☐ Configure Object Lifecycle Management
 - ☐ Set up BigQuery datasets and tables
 - ☐ Deploy Neo4j instance (GCP Marketplace or self-managed)
 - ☐ Deploy Qdrant for vector search
 - ☐ Set up Redis Cache (Memorystore)
- ☐ **Networking**
 - ☐ Configure VPC network
 - ☐ Set up Cloud NAT for outbound connectivity
 - ☐ Configure private service access
 - ☐ Set up load balancers and domains

Agent Development Checklist

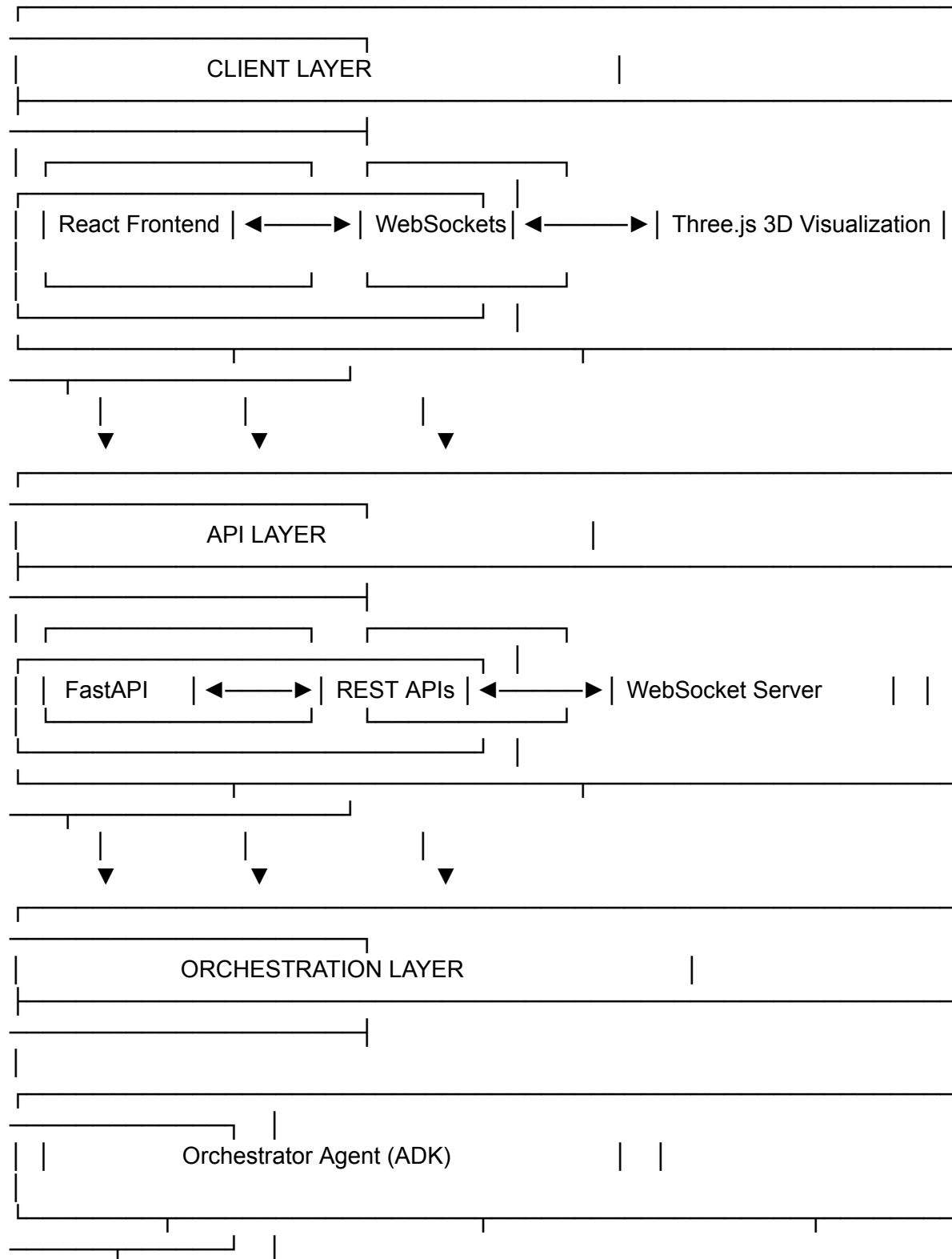
- **[] Ingestion Agent**
 - [] Document AI processor setup
 - [] OCR and layout analysis
 - [] Scientific entity recognition
 - [] Citation extraction
 - [] Figure and table detection
 - [] Storage integration
- **[] Citation Graph Agent**
 - [] Neo4j client integration
 - [] Reference parsing and normalization
 - [] Graph schema definition
 - [] Entity resolution
 - [] Graph metrics computation
 - [] Embedding generation
- **[] Insight Generation Agent**
 - [] BigQuery integration
 - [] Trend analysis queries
 - [] Gemini API integration
 - [] Claim generation
 - [] Insight formatting
- **[] Counteragent (Critic)**
 - [] Verification logic
 - [] Fact-checking against sources
 - [] Alternative perspective generation
 - [] Fallback mechanisms
- **[] 3D Visualizer Agent**
 - [] Three.js integration
 - [] Force-directed layout
 - [] WebSocket streaming
 - [] Interactive controls
 - [] Visual styling
- **[] UI/UX Agent**
 - [] React component generation
 - [] Dashboard layout
 - [] Chart integration
 - [] Filtering controls
 - [] Report generation
- **[] Orchestrator Agent**
 - [] ADK integration
 - [] Workflow definition
 - [] Error handling

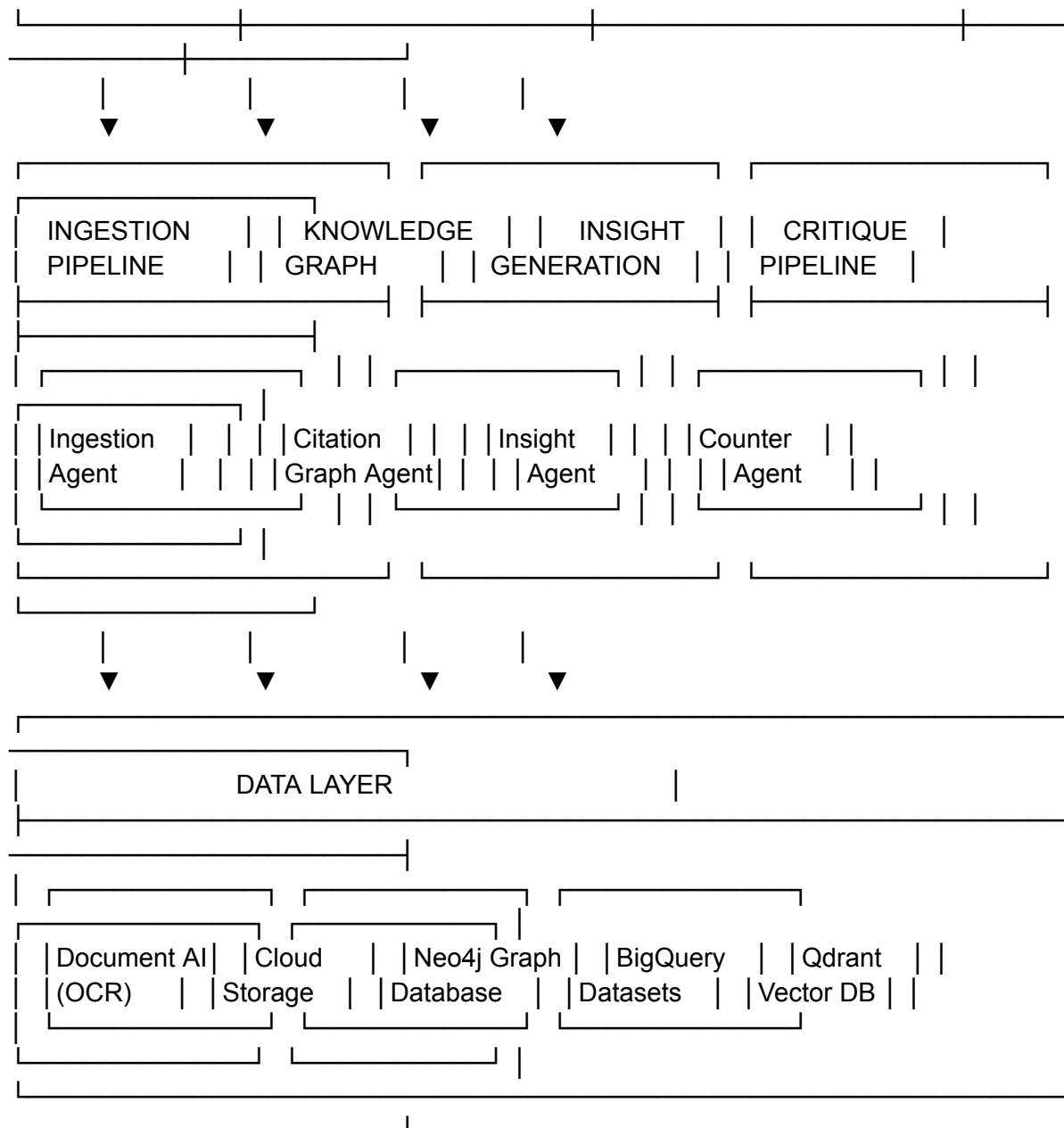
- [] Monitoring setup



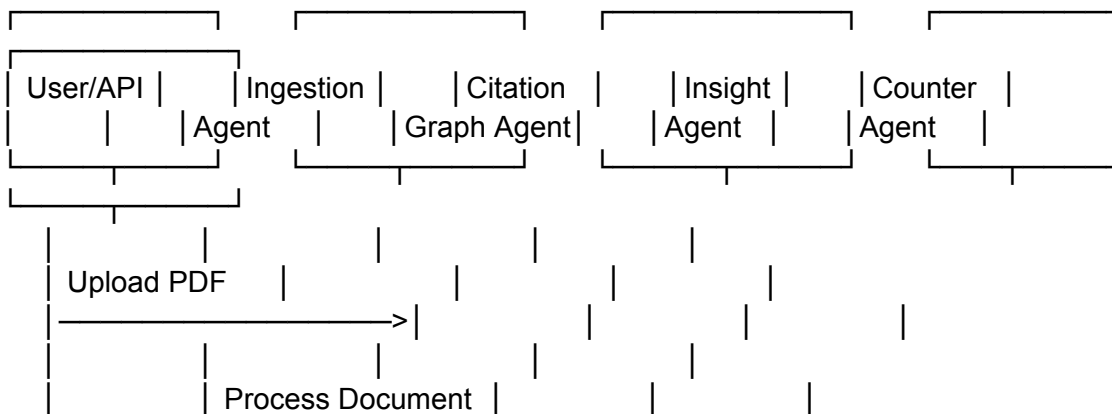
Detailed Architecture Diagrams

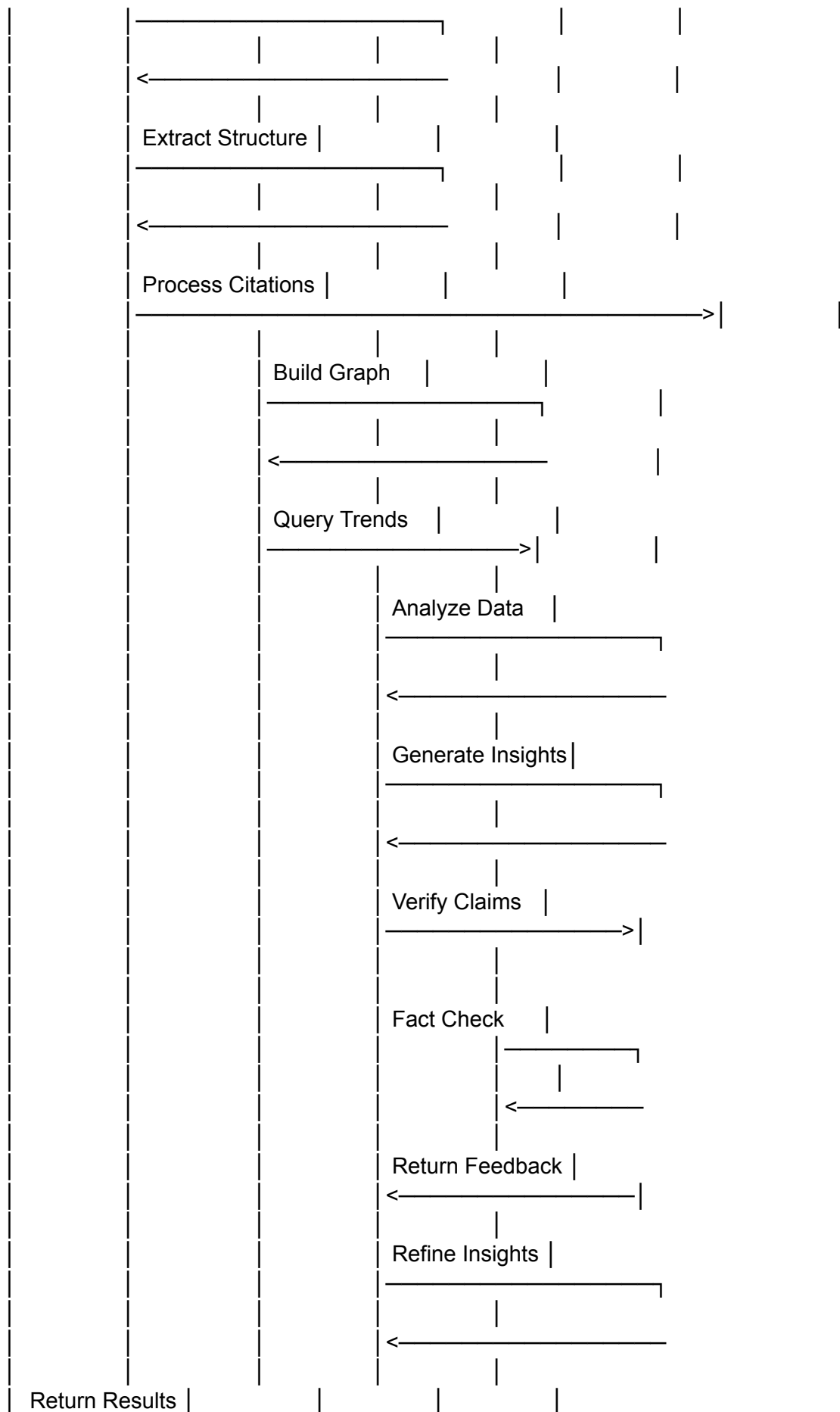
System Interaction Diagram

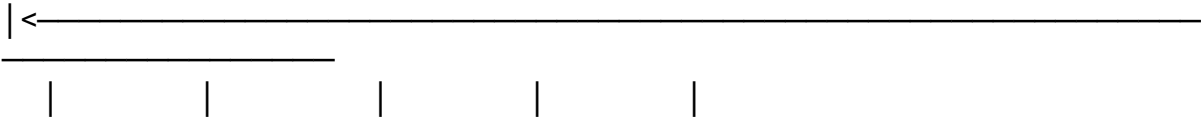




Data Flow Sequence Diagram







Code Samples

Document Processing Pipeline

agents/ingestion/processors/document_pipeline.py

```
import os
from typing import Dict, List, Any
from google.cloud import documentai_v1 as documentai
from google.cloud import storage
import scispacy
import spacy

class DocumentProcessor:
    def __init__(self, project_id, location, processor_id):
        self.documentai_client = documentai.DocumentProcessorServiceClient()
        self.processor_name =
f"projects/{project_id}/locations/{location}/processors/{processor_id}"
        self.storage_client = storage.Client()
        self.nlp = spacy.load("en_core_sci_sm") # SciSpacy model for scientific text

    def process_document(self, gcs_uri: str) -> Dict[str, Any]:
        """Process a PDF document stored in GCS."""
        # Extract document content from GCS
        bucket_name, blob_name = self._parse_gcs_uri(gcs_uri)
        bucket = self.storage_client.bucket(bucket_name)
        blob = bucket.blob(blob_name)
        document_content = blob.download_as_bytes()

        # Process with Document AI
        document = {"content": document_content, "mime_type": "application/pdf"}
        request = {"name": self.processor_name, "raw_document": document}
        result = self.documentai_client.process_document(request=request)
        processed_document = result.document

        # Extract document structure
        document_structure = self._extract_document_structure(processed_document)

        # Process scientific entities
        entities = self._extract_scientific_entities(document_structure)
```

```

# Extract citations
citations = self._extract_citations(document_structure)

# Combine results
document_data = {
    "document_id": os.path.basename(blob_name).split(".")[0],
    "content": document_structure,
    "entities": entities,
    "citations": citations,
    "metadata": self._extract_metadata(processed_document)
}

# Store processed results
self._store_processed_document(document_data, bucket_name)

return document_data

def _parse_gcs_uri(self, gcs_uri: str) -> tuple:
    """Parse GCS URI into bucket and blob name."""
    parts = gcs_uri.replace("gs://", "").split("/", 1)
    bucket_name = parts[0]
    blob_name = parts[1] if len(parts) > 1 else ""
    return bucket_name, blob_name

def _extract_document_structure(self, document) -> Dict[str, Any]:
    """Extract document structure from Document AI output."""
    # Extract text and layout
    text = document.text
    pages = []

    for page in document.pages:
        paragraphs = []
        for paragraph in page.paragraphs:
            paragraph_text = self._get_text_from_layout(text, paragraph.layout)
            paragraphs.append({
                "text": paragraph_text,
                "bounding_box": self._get_bounding_box(paragraph.layout.bounding_poly)
            })

        tables = []
        for table in page.tables:
            table_data = {
                "rows": len(table.header_rows) + len(table.body_rows),
                "columns": max([len(row.cells) for row in table.header_rows + table.body_rows],
default=0),
                "cells": []
            }

```

```

        for row_idx, row in enumerate(table.header_rows + table.body_rows):
            for col_idx, cell in enumerate(row.cells):
                cell_text = self._get_text_from_layout(text, cell.layout)
                table_data["cells"].append({
                    "row": row_idx,
                    "column": col_idx,
                    "text": cell_text,
                    "is_header": row_idx < len(table.header_rows)
                })

            tables.append(table_data)

        pages.append({
            "page_number": page.page_number,
            "paragraphs": paragraphs,
            "tables": tables
        })

    # Try to identify document sections
    sections = self._identify_sections(text, pages)

    return {
        "text": text,
        "pages": pages,
        "sections": sections
    }

def _get_text_from_layout(self, text: str, layout) -> str:
    """Extract text from a layout segment."""
    if layout.text_anchor.text_segments:
        segments = layout.text_anchor.text_segments
        return " ".join([text[s.start_index:s.end_index] for s in segments])
    return ""

def _get_bounding_box(self, bounding_poly) -> Dict[str, float]:
    """Convert a bounding poly to a bounding box dict."""
    return {
        "x": bounding_poly.vertices[0].x,
        "y": bounding_poly.vertices[0].y,
        "width": bounding_poly.vertices[2].x - bounding_poly.vertices[0].x,
        "height": bounding_poly.vertices[2].y - bounding_poly.vertices[0].y
    }

def _identify_sections(self, text: str, pages: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """Identify document sections based on formatting and keywords."""
    # Simple section detection based on paragraph formatting and common headers
    sections = []
    current_section = None

```

```

section_keywords = ["abstract", "introduction", "background", "methods",
                    "methodology", "results", "discussion", "conclusion",
                    "references", "acknowledgments", "appendix"]

# Combine paragraphs from all pages
all_paragraphs = []
for page in pages:
    all_paragraphs.extend([(p, page["page_number"]) for p in page["paragraphs"]])

for paragraph, page_num in all_paragraphs:
    # Check if paragraph looks like a section header
    paragraph_text = paragraph["text"].strip().lower()

    # If very short and ends with common section name, likely a header
    words = paragraph_text.split()
    if (len(words) <= 3 and
        any(paragraph_text.endswith(keyword) for keyword in section_keywords)):
        if current_section:
            sections.append(current_section)

            current_section = {
                "title": paragraph["text"].strip(),
                "content": "",
                "start_page": page_num
            }
        elif current_section:
            if current_section["content"]:
                current_section["content"] += " "
            current_section["content"] += paragraph["text"]
            current_section["end_page"] = page_num

# Add the last section
if current_section:
    sections.append(current_section)

return sections

def _extract_scientific_entities(self, document_structure: Dict[str, Any]) -> List[Dict[str, Any]]:
    """Extract scientific entities using SciSpacy."""
    entities = []

    # Process text with SciSpacy
    doc = self.nlp(document_structure["text"])

    for ent in doc.ents:
        entity = {
            "text": ent.text,

```

```

        "label": ent.label_,
        "start": ent.start_char,
        "end": ent.end_char
    }
    entities.append(entity)

```

```

return entities

```

```

def _extract_citations(self, document_structure: Dict[str, Any]) -> List[Dict[str, Any]]:

```

```

    """Extract citations from document."""

```

```

    citations = []

```

```

    text = document_structure["text"]

```

```

    # Look for common citation patterns

```

```

    # Pattern 1: [number]

```

```

    import re

```

```

    bracket_citations = re.finditer(r'\[(\d+)\]', text)

```

```

    for match in bracket_citations:

```

```

        citation = {
            "type": "numeric",
            "citation_text": match.group(0),
            "citation_number": match.group(1),
            "start": match.start(),
            "end": match.end()
        }
        citations.append(citation)

```

```

    # Pattern 2: Author et al., Year

```

```

    author_year_pattern =

```

```

    re.compile(r'([A-Z][a-z]+(?:\s+et\s+al\.\s+and\s+[A-Z][a-z]+)?),?\s+(\d{4})?')

```

```

    author_citations = author_year_pattern.finditer(text)

```

```

    for match in author_citations:

```

```

        citation = {
            "type": "author_year",
            "citation_text": match.group(0),
            "start": match.start(),
            "end": match.end()
        }
        citations.append(citation)

```

```

    # Extract references section

```

```

    references_section = None

```

```

    for section in document_structure["sections"]:

```

```

        if "reference" in section["title"].lower():
            references_section = section
            break

```

```

    if references_section:

```

```

# Process reference list
reference_entries = self._parse_references(references_section["content"])
for i, entry in enumerate(reference_entries):
    # Try to link citations to references
    for citation in citations:
        if citation["type"] == "numeric" and int(citation["citation_number"]) == i + 1:
            citation["reference"] = entry

return citations

def _parse_references(self, references_text: str) -> List[str]:
    """Parse reference list into individual entries."""
    import re

    # Try to split by common reference numbering patterns
    # Pattern: [1] or 1. or 1
    splits = re.split(r'\n\s*(?:\[\d+\]|\d+\.\d+)\s+', '\n' + references_text)
    if len(splits) > 1:
        # First element is empty due to the leading \n
        return [ref.strip() for ref in splits[1:]]

    # Fallback: try to split by newlines with heuristics
    lines = references_text.split('\n')
    references = []
    current_ref = ""

    for line in lines:
        line = line.strip()
        if not line:
            continue

        # Heuristic: If line starts with author-like pattern, it's likely a new reference
        if re.match(r'^[A-Z][a-z]+\s+[A-Z]\.', line):
            if current_ref:
                references.append(current_ref)
            current_ref = line
        else:
            if current_ref:
                current_ref += " " + line
            else:
                current_ref = line

    if current_ref:
        references.append(current_ref)

    return references

def _extract_metadata(self, document) -> Dict[str, Any]:

```



```

"""Extract document metadata."""
metadata = {}

# Extract document entities that might contain metadata
for entity in document.entities:
    entity_type = entity.type_
    mention_text = self._get_text_from_layout(document.text, entity.text_anchor)

    if entity_type in ["TITLE", "AUTHOR", "PUBLICATION_DATE", "DOI"]:
        metadata[entity_type.lower()] = mention_text

return metadata

def _store_processed_document(self, document_data: Dict[str, Any], source_bucket: str)
-> str:
    """Store processed document data in Cloud Storage."""
    import json

    # Create a JSON string
    json_data = json.dumps(document_data)

    # Define target location
    document_id = document_data["document_id"]
    target_bucket_name = f"{source_bucket}-processed"
    target_blob_name = f"{document_id}/processed.json"

    # Ensure target bucket exists
    try:
        target_bucket = self.storage_client.bucket(target_bucket_name)
        if not target_bucket.exists():
            target_bucket = self.storage_client.create_bucket(target_bucket_name)
    except Exception as e:
        # Fall back to source bucket with different path
        target_bucket = self.storage_client.bucket(source_bucket)
        target_blob_name = f"processed/{document_id}/processed.json"

    # Upload processed data
    target_blob = target_bucket.blob(target_blob_name)
    target_blob.upload_from_string(json_data, content_type="application/json")

    return f"gs://{target_bucket.name}/{target_blob_name}"

```

Knowledge Graph Construction

agents/citation_graph/graph_builder/neo4j_builder.py

from typing import Dict, List, Any

```

import uuid
from neo4j import GraphDatabase
import logging

class Neo4jGraphBuilder:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
        self.logger = logging.getLogger(__name__)
        self._initialize_schema()

    def _initialize_schema(self):
        """Initialize Neo4j schema with constraints and indexes."""
        with self.driver.session() as session:
            # Create constraints for uniqueness and indexing
            queries = [
                "CREATE CONSTRAINT paper_id IF NOT EXISTS FOR (p:Paper) REQUIRE p.id IS UNIQUE",
                "CREATE CONSTRAINT author_id IF NOT EXISTS FOR (a:Author) REQUIRE a.id IS UNIQUE",
                "CREATE CONSTRAINT concept_id IF NOT EXISTS FOR (c:Concept) REQUIRE c.id IS UNIQUE",
                "CREATE CONSTRAINT institution_id IF NOT EXISTS FOR (i:Institution) REQUIRE i.id IS UNIQUE",
                "CREATE INDEX paper_title IF NOT EXISTS FOR (p:Paper) ON (p.title)",
                "CREATE INDEX author_name IF NOT EXISTS FOR (a:Author) ON (a.name)",
                "CREATE INDEX concept_name IF NOT EXISTS FOR (c:Concept) ON (c.name)"
            ]

            for query in queries:
                try:
                    session.run(query)
                except Exception as e:
                    self.logger.warning(f"Error creating constraint/index: {e}")

    def process_document(self, document_data: Dict[str, Any]) -> str:
        """Process document data and build graph."""
        document_id = document_data["document_id"]
        self.logger.info(f"Processing document: {document_id}")

        with self.driver.session() as session:
            # Create paper node
            paper_id = session.write_transaction(
                self._create_paper_node,
                document_id,
                document_data
            )

            # Process authors

```

```

authors = self._extract_authors(document_data)
for author in authors:
    author_id = session.write_transaction(
        self._create_author_node,
        author
    )

    # Connect author to paper
    session.write_transaction(
        self._create_authorship_relation,
        author_id,
        paper_id
    )

    # Process institutions
    if "affiliation" in author:
        institution_id = session.write_transaction(
            self._create_institution_node,
            author["affiliation"]
        )

        # Connect author to institution
        session.write_transaction(
            self._create_affiliation_relation,
            author_id,
            institution_id
        )

# Process concepts
concepts = self._extract_concepts(document_data)
for concept in concepts:
    concept_id = session.write_transaction(
        self._create_concept_node,
        concept
    )

    # Connect paper to concept
    session.write_transaction(
        self._create_mentions_relation,
        paper_id,
        concept_id,
        concept["relevance"]
    )

# Process citations
citations = document_data.get("citations", [])
for citation in citations:
    if "reference" in citation:

```

```

        # Try to resolve cited paper
        cited_paper_id = self._resolve_reference(session, citation["reference"])
        if cited_paper_id:
            # Create citation relationship
            session.write_transaction(
                self._create_citation_relation,
                paper_id,
                cited_paper_id,
                citation.get("citation_text", "")
            )

    return paper_id

def _extract_authors(self, document_data: Dict[str, Any]) -> List[Dict[str, Any]]:
    """Extract authors from document data."""
    authors = []

    # Try to get authors from metadata
    if "metadata" in document_data and "author" in document_data["metadata"]:
        author_text = document_data["metadata"]["author"]
        author_names = [name.strip() for name in author_text.split(",")]

        for name in author_names:
            author = {"name": name}
            authors.append(author)

    # Look for authors in entities
    if "entities" in document_data:
        for entity in document_data["entities"]:
            if entity["label"] == "PERSON" and entity["text"] not in [a["name"] for a in authors]:
                # Check if this might be an author (appears early in document)
                if entity["start"] < len(document_data["content"]["text"]) / 4:
                    author = {"name": entity["text"]}
                    authors.append(author)

    return authors

def _extract_concepts(self, document_data: Dict[str, Any]) -> List[Dict[str, Any]]:
    """Extract concepts from document data."""
    concepts = []
    concept_frequency = {}

    # Extract from entities
    if "entities" in document_data:
        for entity in document_data["entities"]:
            if entity["label"] in ["ORG", "GPE", "FACILITY", "CHEMICAL", "DISEASE",
                                   "PROCEDURE"]:
                concept_text = entity["text"].lower()

```

```

        if concept_text not in concept_frequency:
            concept_frequency[concept_text] = 1
        else:
            concept_frequency[concept_text] += 1

# Convert frequencies to concepts with relevance scores
total_mentions = sum(concept_frequency.values()) or 1 # Avoid division by zero
for concept_text, frequency in concept_frequency.items():
    relevance = frequency / total_mentions
    concepts.append({
        "name": concept_text,
        "type": "ENTITY",
        "relevance": relevance
    })

return concepts

def _resolve_reference(self, session, reference_text: str) -> str:
    """Try to resolve a reference to an existing paper."""
    # Extract DOI if present
    import re
    doi_match = re.search(r'10\.\d{4,9}/[-._;()/:A-Z0-9]+', reference_text, re.IGNORECASE)
    if doi_match:
        doi = doi_match.group(0)
        result = session.run(
            "MATCH (p:Paper {doi: $doi}) RETURN p.id AS paper_id",
            doi=doi
        )
        record = result.single()
        if record:
            return record["paper_id"]

    # Try title-based matching
    # Extract potential title (text between quotes or first sentence)
    title_match = re.search(r'"([^\"]+)"', reference_text)
    if title_match:
        potential_title = title_match.group(1)
        result = session.run(

```